

Optimization Sequence — Throughput & Accuracy Journey

Step-by-step changes on SmolVLA + LeRobot (GPU / Strix)

#	Step	What changed	Impact
1	Diffusion steps ↓	Reduced denoising steps 30 → 2 (extra steps shown unnecessary)	2 → 4.5–5 inf/s
2	torch.compile	Rewrote LeRobot SmolVLA modules so the full flow forms a single graph → optimized GPU kernels	→ 8.5 inf/s
3	fp32 → fp16	Cast weights to fp16 <i>before</i> compile so the graph has no up/down-scale ops around each layer	→ 13–15 inf/s
4	RTC on (Real-Time Chunking)	Camera pipeline + model run on parallel threads sharing the GPU → reactive end-to-end pipeline	Reactivity ↑
5	chunk_size_threshold ↓	Next inference triggered after only 5 consumed actions → fresher images drive predictions	Reactivity ↑
6	Action smoothing	Post-processing removes jitter/shake in continuous-action output	Smoothness ↑
7	More data + fine-tune	Scaled training data and fine-tuned model	MAE 6 → ~3

Bottom line: ~7× throughput (2 → 13–15 inf/s) and ~50% lower MAE (6 → ~3, usable limit ~3.5) — model is now usable for live gesture mimic on Strix.

Throughput → Accuracy Coupling

Faster model = more accurate, more responsive robot

Why a slow model hurts accuracy

- **Drops camera frames** → operator ↔ robot sync is lost.
- Forced to predict **far into the future** from stale observations — long-horizon predictions are inherently unreliable.
- Lag accumulates → robot mimics what the operator did *seconds ago*.

Why a fast model is more accurate

- Consumes **every fresh frame** from the camera stream.
- New predictions **overwrite** old ones, always grounded in current state → less guessing.
- Short, reliable horizons → tighter operator ↔ robot sync.

Key insight for leadership: throughput is not just speed — it directly raises **behavioral accuracy** by keeping predictions grounded in the latest observation.